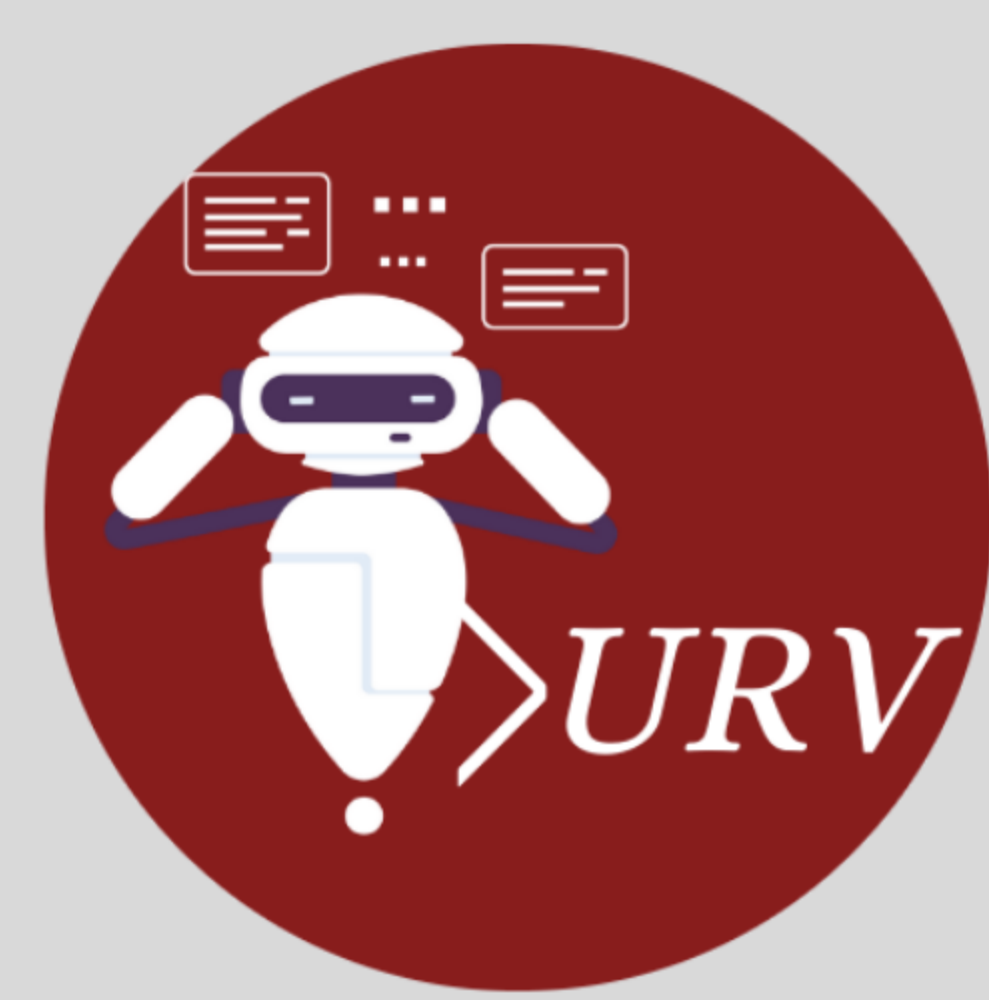




Generating Linearly Separable SAT Instances

Quinn Mayo, Rahul Vedula, Rachel Lahav, Landrick Diaz

PhD Mentor: Kyle Doney



Introduction

The **SAT** problem asks if a given boolean formula is satisfiable. We explore the subset of linearly separable **SAT** formulas which are conjectured to be more easily decidable than arbitrary **SAT** instances. We develop a generator for these formulas and provide code to convert them to graph structures for a graph neural network.

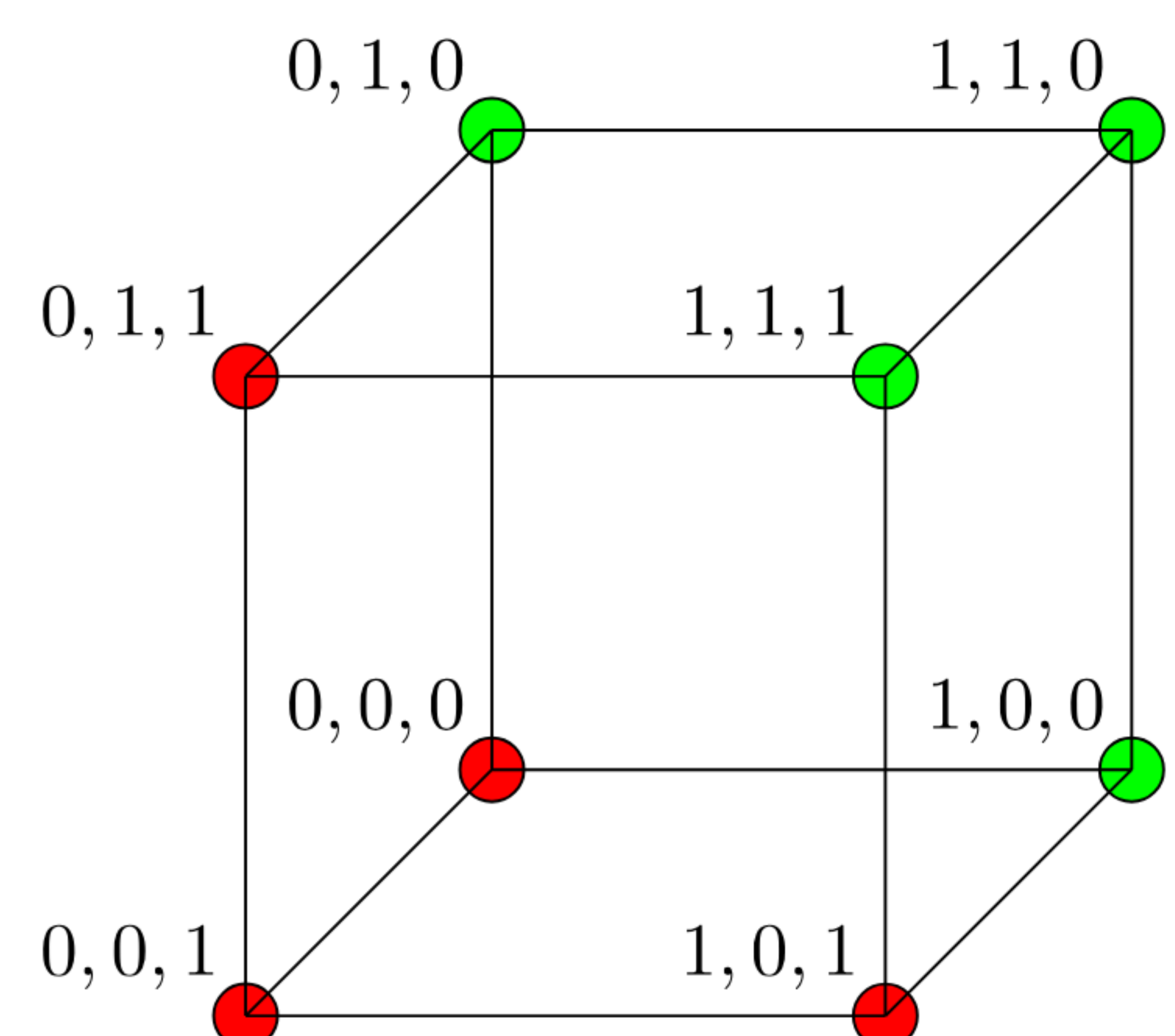


Fig. 1.1: A linearly separable boolean function.

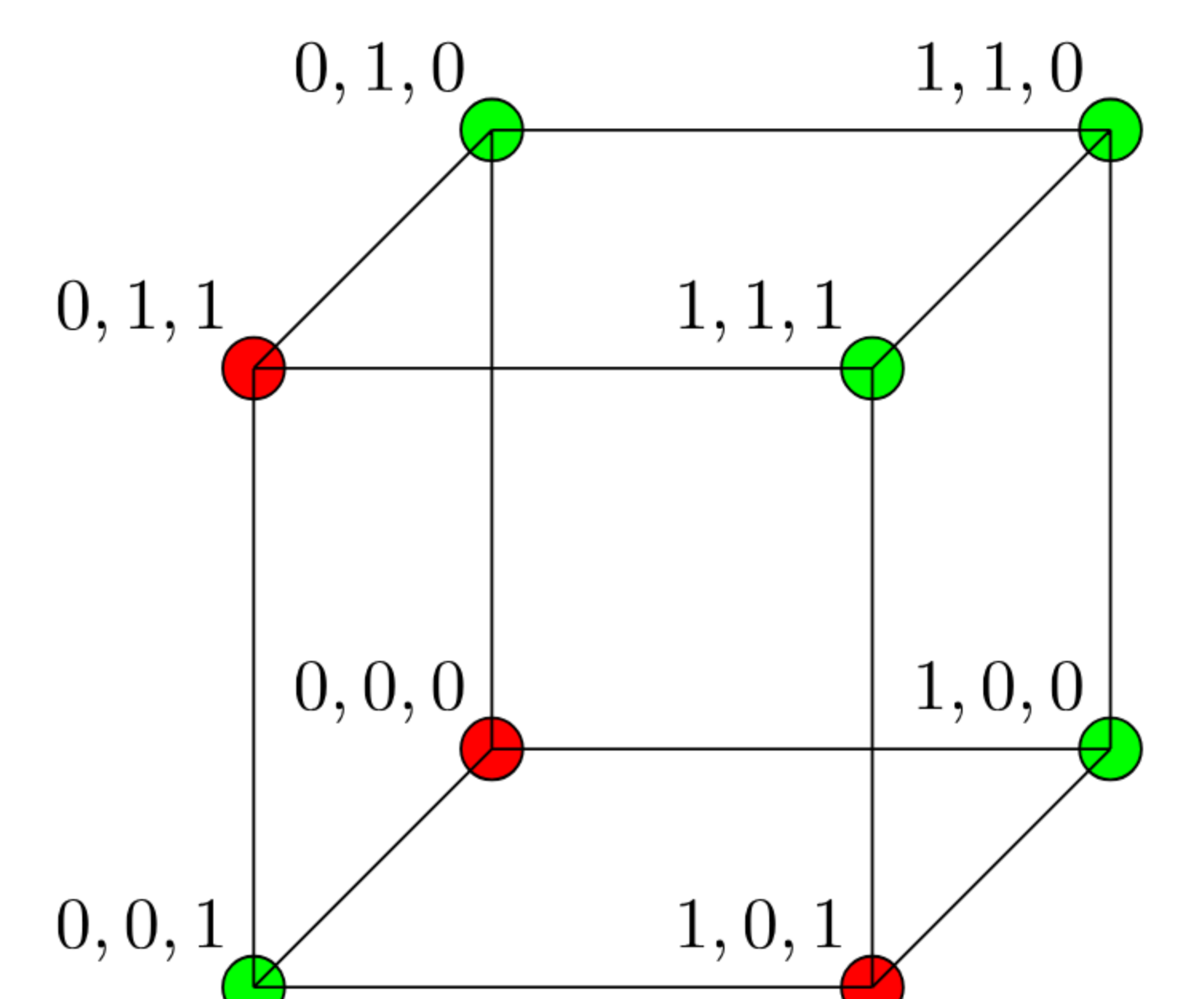


Fig. 1.2: A linearly non-separable boolean function.

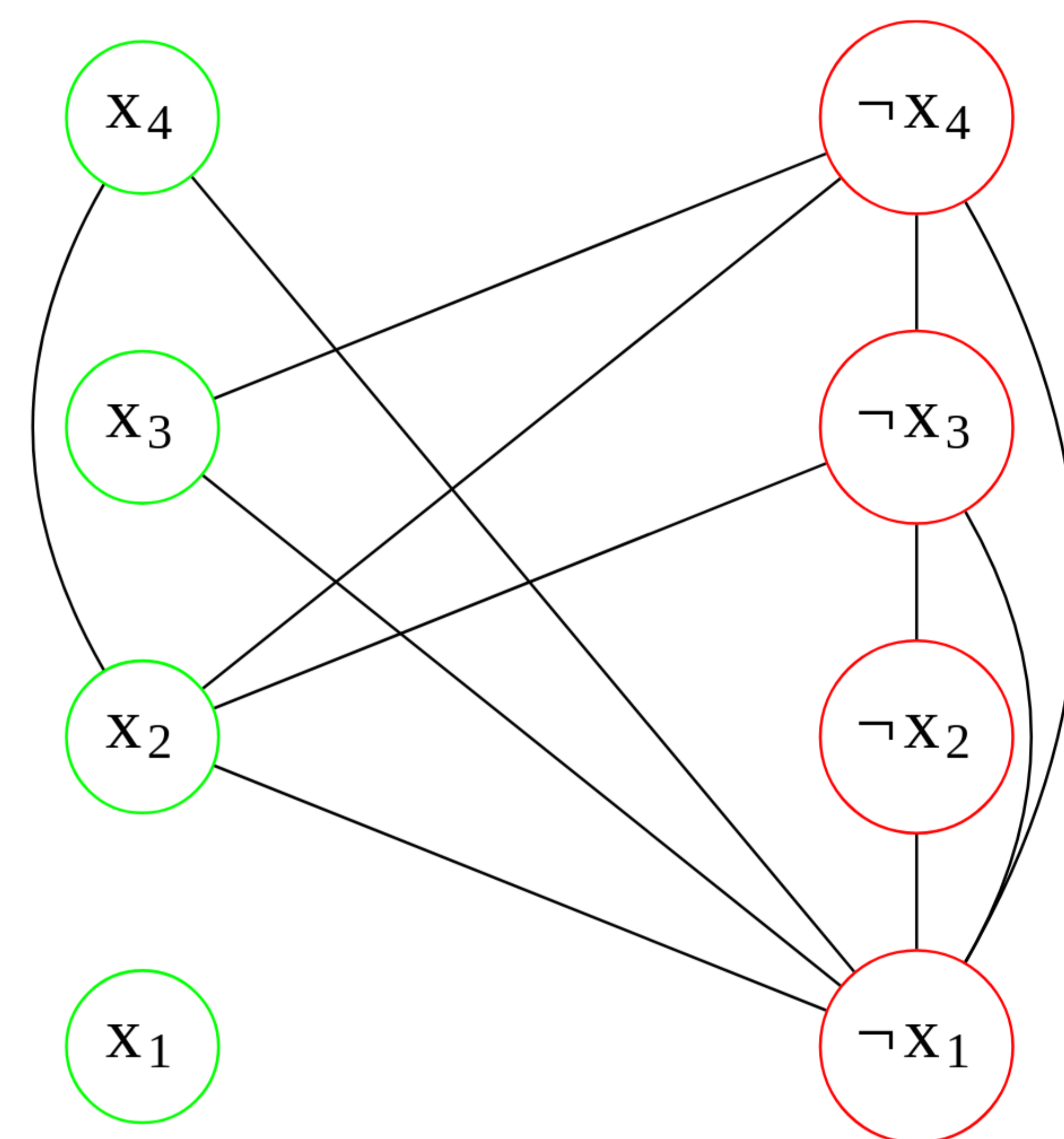


Fig 3: Variable-to-variable graph of linearly separable boolean function
 $(\neg X_1 \vee X_3 \vee \neg X_4) \wedge (\neg X_1 \vee \neg X_2 \vee \neg X_3) \wedge (\neg X_1 \vee X_2 \vee X_4) \wedge (X_2 \vee \neg X_3 \vee \neg X_4)$

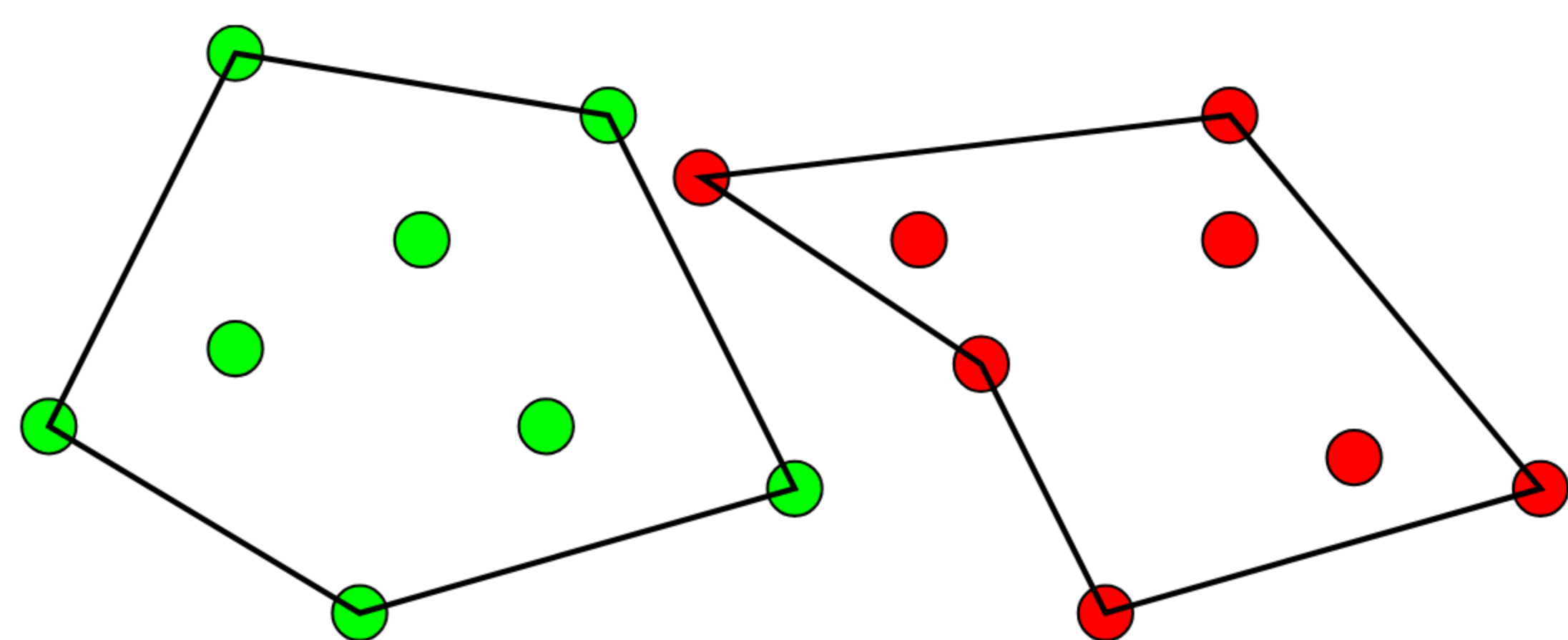


Fig 2.1: Convex hull test on linearly separable points.

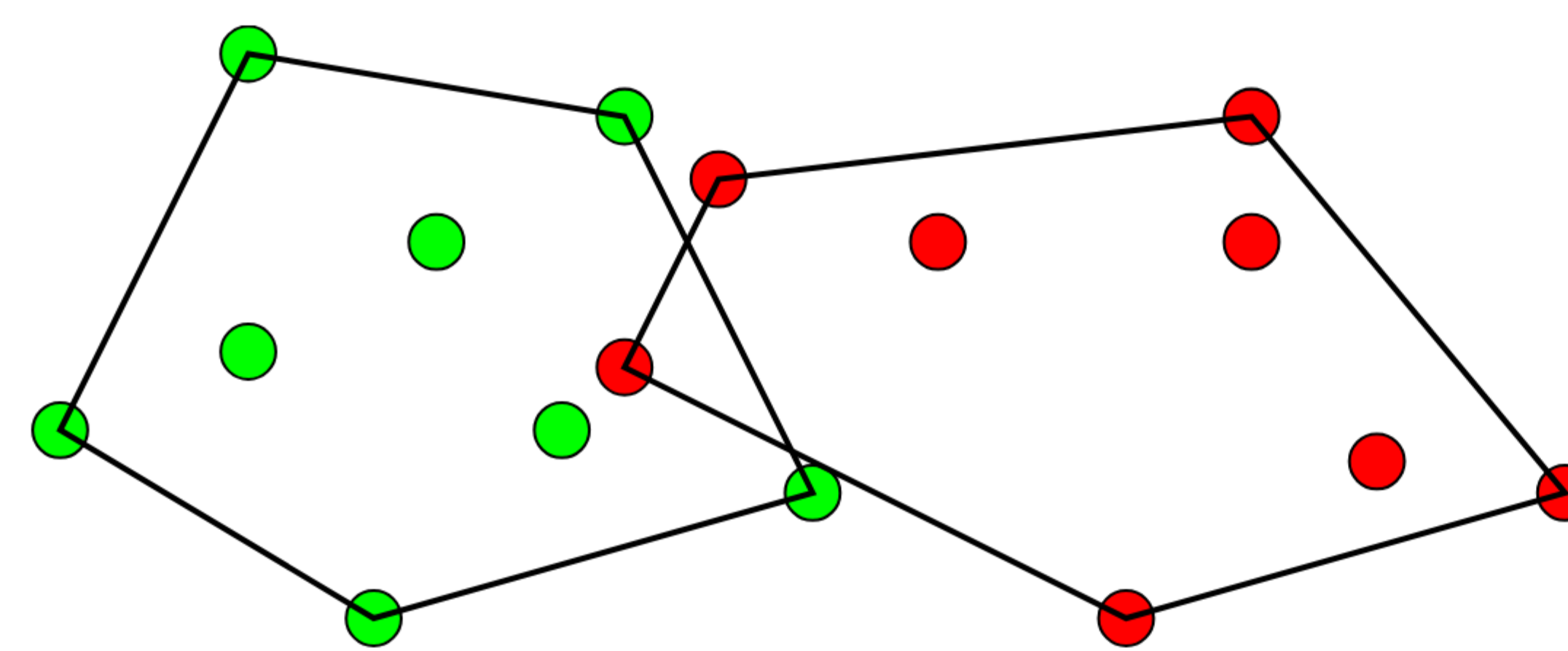


Fig 2.2: Convex hull test on linearly non-separable points.

Motivation

LINSEPSAT captures our interest due to its potential computational simplicity relative to general **SAT** problems. **LINSEPSAT** has strong similarities to **UNIQUESAT**; another notable subset of **SAT** instances. Additionally, we believe **LINSEPSAT** is a natural problem for neural networks to decide as linear classifiers are good at learning linearly separable data. Given the generally poor performance of neural network models at solving **NP**-complete problems, demonstrating that neural networks perform well on **LINSEPSAT** would show that either (1) **LINSEPSAT** is not likely **NP-hard**, or (2) **LINSEPSAT** is **NP-hard** and therefore is a good format for neural networks to learn **NP-hard** problems.

Definitions and Facts

- Two sets of n -dimensional points are *linearly separable* if there exists a $(n - 1)$ -dimensional hyperplane separating them.
 - A boolean function of n variables ϕ is called *linearly separable* if the 2-coloring of the n -dimensional hypercube induced by ϕ is linearly separable (see Fig. 1.1 and 1.2).
 - Linear classifiers can learn exactly those boolean functions which are linearly separable.
- A *promise problem* is a special type of decision problem where the input is "promised" to have a particular property. We consider the promise problems:

$\text{LINSEPSAT} := \{\phi \in \text{SAT} : \phi \text{ is linearly separable}\}$
 $\text{UNIQUESAT} := \{\phi \in \text{SAT} : \phi \text{ has at most 1 satisfying assignment}\}$

 - UNIQUESAT** poly-time reduces to **LINSEPSAT**.
 - UNIQUESAT** is in the complexity class $\text{UP} \subseteq \text{NP}$, the subset of **NP** problems with one accepting computation path.
 - It is not known if **LINSEPSAT** is **NP-hard** or in **UP**.
- A boolean function is in *conjunctive normal form* (CNF) if it is written as a conjunction (AND) of *clauses*, which are a disjunction (OR) of boolean variables.

Methodology

The Convex Hull Test: The convex hull test states that if the convex hulls of two point sets intersect, then the points are not linearly separable (Fig. 2.1, 2.2). To generate linearly separable **SAT** instances, we randomly generate a boolean function, construct the corresponding boolean hypercube, and then run the convex hull test on the sets of true points and false points.

In practice, we were only able to generate **LINSEPSAT** instances of up to 6 variables; it quickly becomes prohibitive to find linearly separable formulas since testing the linear separability of a boolean function is **co-NP-hard** [2].

CNF Graphs & Graph Neural Networks (GNN): Graphs are a desirable way to encode **SAT** formulas since they preserve semantic content, unlike syntax trees [1]. Given a CNF **SAT** instance, we create a variable-to-variable graph where two variables are connected if they appear in the same clause (Fig. 3). We can then train a GNN to classify **LINSEPSAT** graphs as satisfiable or not, and compare the performance with a GNN trained on **SAT** graphs.

Citations

- Bünz, Bebedikt, & Lamm, M. (2017). Graph Neural Networks and Boolean Satisfiability.
- Jiří Šíma, Pekka Orponen; General-Purpose Computation with Neural Networks: A Survey of Complexity Theoretic Results. *Neural Comput* 2003.